

Inbound Email Receiving

This document describes how inbound emails (replies from learners/users to platform-sent emails) are received, parsed, and stored, and what infrastructure must be provisioned in AWS before turning the feature on.

TL;DR

- Inbound emails arrive at SES (us-east-1) → saved to S3 → S3 event triggers SQS → notification service consumes the message and writes an **INBOUND_EMAIL** row to **notification_log**, linked to the original outbound email by **In-Reply-To** header.
- The student communication timeline already supports **INBOUND** direction. New inbound emails appear automatically with a "Received via Email" label.
- Feature is **off by default**. Enable by setting **INBOUND_EMAIL_ENABLED=true** once AWS infra is ready.

Architecture



Code Changes

New files

File	Purpose
<code>src/main/resources/db/migration/V24__Create_email_address_mapping.sql</code>	Creates <code>email_address_mapping</code> (email → institute lookup)
<code>features/notification_log/entity/EmailAddressMapping.java</code>	JPA entity for the mapping table
<code>features/notification_log/repository/EmailAddressMappingRepository.java</code>	Repository with <code>findByEmailAddressAnd</code> + Postgres ON CONFLICT up

File	Purpose
<code>config/AwsInboundConfig.java</code>	Bean config for inbound S3 (v2 SDK) clients pointing at the region
<code>service/SqsInboundEmailListener.java</code>	<code>@SqsListener</code> consuming queue, parses S3 event JSON service
<code>service/InboundEmailService.java</code>	Core: download from S3, parse, dedup, rate-limit, link replies,

Modified files

File	Change
<code>service/EmailService.java</code>	After <code>mailSender.send</code> captures <code>mimeMessage.getMessageId</code> and stores it as <code>sourceId</code> outbound <code>notification</code> row. This enables reply links for new emails. OTP path unchanged (its <code>sourceId</code> already used for service identification).
<code>features/announcements/service/EmailConfigurationService.java</code>	When an institute adds an config, also upserts a row <code>email_address_mapping</code> inbound replies addressed to mailbox can be routed to the correct institute.
<code>features/communication_timeline/service/CommunicationTimelineService.java</code>	Added <code>INBOUND_EMAIL</code> type → channel/direction mapping. <code>EMAIL</code> channel now resolves <code>[EMAIL, INBOUND_EMAIL]</code> direction-based filtering; new <code>mapInboundEmailFields</code> sets status <code>RECEIVED</code> and sender/recipient from the <code>messagePayload</code> .
<code>application-{dev,stage,prod}.properties</code>	New properties added (see Configuration).
<code>.github/workflows/maven-publish-notification-service.yml</code>	New env vars wired through build + Kubernetes deployment
<code>.github/workflows/vet-deploy-notification-service.yml</code>	Same.

Frontend

No changes required. `student-communication-timeline.tsx` already handles `direction === 'INBOUND'` and `status === 'RECEIVED'` for emails (green left-border, ✓ arrow, "Received via Email" label).

Feature Flag

The new functionality is gated by a dedicated opt-in flag — independent of the existing `aws.sqs.enabled` (which controls outbound SES event tracking).

```
aws.inbound.email.enabled=${INBOUND_EMAIL_ENABLED:false}
```

When `false` (the default):

- `AwsInboundConfig`, `SqsInboundEmailListener`, `InboundEmailService` beans are **not loaded**.
- Zero AWS calls. Zero log noise. The service starts identically to before.
- The only always-active changes are: the `Message-ID` capture in `EmailService` (purely additive — `sourceId` was previously `null` for outbound emails) and the upsert into `email_address_mapping` whenever an email config is saved (wrapped in try-catch).

This means the code is **safe to merge and deploy before the AWS infrastructure exists**.

Configuration

Spring properties (already added to all `application-*.properties`)

```
# Master flag — keep false until AWS infra is ready
aws.inbound.email.enabled=${INBOUND_EMAIL_ENABLED:false}

# Inbound region must be one of: us-east-1, us-west-2, eu-west-1
# (SES inbound is not available in ap-south-1 or other regions)
aws.inbound.region=${INBOUND_EMAIL_AWS_REGION:us-east-1}
aws.inbound.accessKey=${INBOUND_EMAIL_AWS_ACCESS_KEY:}
aws.inbound.secretKey=${INBOUND_EMAIL_AWS_SECRET_KEY:}
aws.inbound.sqs.queue-name=${INBOUND_EMAIL_SQS_QUEUE_NAME:vacademy-inbound-emails-queue}
aws.s3.inbound-email-bucket=${INBOUND_EMAIL_S3_BUCKET:vacademy-inbound-emails}
```

GitHub Actions secrets to add

Secret	Example value	Notes
<code>INBOUND_EMAIL_ENABLED</code>	<code>false</code> (initially) → <code>true</code> (after AWS setup)	Master switch
<code>INBOUND_EMAIL_AWS_REGION</code>	<code>us-east-1</code>	Must be a SES-inbound-supported region
<code>INBOUND_EMAIL_AWS_ACCESS_KEY</code>	<code>AKIA...</code>	IAM user (see policy below)
<code>INBOUND_EMAIL_AWS_SECRET_KEY</code>	<code>...</code>	Same IAM user secret
<code>INBOUND_EMAIL_SQS_QUEUE_NAME</code>	<code>vacademy-inbound-emails-queue</code>	Queue name only (not URL)
<code>INBOUND_EMAIL_S3_BUCKET</code>	<code>vacademy-inbound-emails</code>	Bucket name

The workflow files already reference all six secrets — until they're set, they pass empty strings, which the properties handle safely.

AWS Setup — Step by Step

Important: All inbound infrastructure must be in **us-east-1** (or us-west-2 / eu-west-1). Existing outbound SES is in ap-south-1 — that region does **not** support inbound. The two are independent and can coexist.

1. Verify domain in SES (us-east-1)

In the AWS Console, switch to **us-east-1**, then:

1. Go to **SES → Verified identities → Create identity**
2. Add domain **vacademy.io** (and any other domain you want to receive replies on)
3. AWS will give you a TXT record + DKIM CNAMEs — add them to your DNS

2. Add MX record

For each domain you want to receive mail at, add this MX record to DNS:

```
Type:      MX
Host:      @                (or whatever subdomain)
Priority: 10
Value:     inbound-smtp.us-east-1.amazonaws.com
```

3. Create S3 bucket

```
Bucket name: vacademy-inbound-emails
Region:      us-east-1
```

Attach this bucket policy so SES can write incoming mail to it:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ses.amazonaws.com" },
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::vacademy-inbound-emails/*",
      "Condition": {
        "StringEquals": {
          "aws:Referer": "<YOUR_AWS_ACCOUNT_ID>"
        }
      }
    }
  ]
}
```

Replace **<YOUR_AWS_ACCOUNT_ID>** with your 12-digit AWS account number.

(Optional but recommended) Add a lifecycle rule to delete objects older than 30 days — once we've parsed and stored, we don't need the raw **.eml** anymore.

4. Create SQS queue

```
Queue name: vacademy-inbound-emails-queue
Region:     us-east-1
Type:       Standard
```

Attach this queue access policy so the S3 bucket can publish events to it:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "s3.amazonaws.com" },
      "Action": "sqs:SendMessage",
      "Resource": "arn:aws:sqs:us-east-1:<YOUR_AWS_ACCOUNT_ID>:vacademy-inbound-emails-queue",
      "Condition": {
        "ArnLike": {
          "aws:SourceArn": "arn:aws:s3:::vacademy-inbound-emails"
        }
      }
    }
  ]
}
```

5. Wire S3 → SQS event notification

In the S3 console:

1. Open the **vacademy-inbound-emails** bucket
2. **Properties** → **Event notifications** → **Create event notification**
3. Configure:
 - Name: **inbound-email-to-sqs**
 - Event types: **All object create events** (**s3:ObjectCreated:***)
 - Destination: **SQS queue** → **vacademy-inbound-emails-queue**

6. Create SES receipt rule

In the SES console (still in us-east-1):

1. **Email receiving** → **Rule sets** → **Create rule set** (if none exists)
2. **Create rule** within the rule set:
 - Recipients: **vacademy.io** (and any other verified domains)
 - Action: **Deliver to S3 bucket**
 - Bucket: **vacademy-inbound-emails**
 - Object key prefix: leave blank (or use **inbound/** if you prefer)
3. **Activate the rule set** (only one rule set can be active at a time)

7. Create / update IAM user

The notification service needs an IAM user (or role) with permissions to read S3 + consume SQS.

Recommended: create a dedicated IAM user for inbound, separate from the existing **SQS_AWS_*** user.

Attach this inline policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::vacademy-inbound-emails/*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "sqs:ReceiveMessage",
        "sqs:DeleteMessage",
        "sqs:GetQueueAttributes",
        "sqs:GetQueueUrl"
      ]
    }
  ]
}
```

```
    ],  
    "Resource": "arn:aws:sqs:us-east-1:<YOUR_AWS_ACCOUNT_ID>:vacademy-inbound-emails-queue"  
  }  
]  
}
```

Generate access keys for this user → put them in GitHub secrets as **INBOUND_EMAIL_AWS_ACCESS_KEY** and **INBOUND_EMAIL_AWS_SECRET_KEY**.

Going Live

Once all 7 AWS steps are done and secrets are in GitHub:

1. Set **INBOUND_EMAIL_ENABLED=true** in GitHub secrets (or directly in the deployment env)
 2. Trigger a deploy (push to **main** or re-run the workflow)
 3. Verify in pod logs:
 - No errors at startup mentioning **inbound**, **S3**, or **SQS**
 - Look for **SqsInboundEmailListener** registration log line
-

Verification

Step A — confirm Message-ID capture (works regardless of inbound flag)

After deploying the code, send any email through the platform. Then in the database:

```
SELECT id, channel_id, source, source_id, notification_date  
FROM notification_log  
WHERE notification_type = 'EMAIL'  
ORDER BY notification_date DESC  
LIMIT 5;
```

source_id should now be populated with values like **<random-id@email.amazonses.com>** — previously it was always NULL for non-OTP emails.

Step B — confirm institute mapping is being populated

After saving an email config in Settings, check:

```
SELECT * FROM email_address_mapping;
```

There should be a row matching the email address you just configured.

Step C — full end-to-end inbound flow

Once **INBOUND_EMAIL_ENABLED=true**:

1. From the platform, send an email to a real mailbox you control
2. Reply to that email
3. Within ~30 seconds, check S3:

```
aws s3 ls s3://vacademy-inbound-emails/ --recursive
```

You should see a new object.

4. Check SQS for any messages currently in flight:

```
aws sqs get-queue-attributes \  
  --queue-url <queue-url> \  
  --attribute-names ApproximateNumberOfMessages
```

5. Check **notification_log**:

```
SELECT id, notification_type, channel_id, source, source_id, user_id, body  
FROM notification_log  
WHERE notification_type = 'INBOUND_EMAIL'  
ORDER BY notification_date DESC  
LIMIT 5;
```

- **channel_id** = sender's email address
- **source** = the **id** of the original outbound email log (reply linking worked)
- **user_id** = same as the original outbound email's **user_id**
- **source_id** = the inbound email's Message-ID (used for dedup)

6. Open the student's communication timeline in the admin dashboard. The reply should appear with green left-border, ✓ arrow, status "Received".

Step D — confirm safe failure modes

- **Wrong creds / non-existent queue:** SQS polling logs **QueueDoesNotExist** or auth errors but the service stays up. No impact on outbound emails.
- **Email to an unknown address:** Discarded with debug log (**No email_address_mapping found for To=...**). No DB row created.
- **Duplicate SQS delivery:** Second delivery is detected via **source_id** dedup and skipped silently.

Edge Cases & Limits

Case	Handling
Spam / unmapped recipient	Discarded — only emails to addresses in email_address_mapping are stored
Duplicate SQS delivery	Dedup on source_id = Message-ID ; second delivery skipped
No In-Reply-To header	Stored standalone; source = null , userId resolved from most recent outbound email to the sender
Multiple To/CC recipients	First match in email_address_mapping wins; one log entry per inbound email
Body > 64 KB	Truncated with [TRUNCATED] marker; full body still stored in messagePayload JSON up to the limit
HTML-only body	Tags stripped for the body field; raw HTML preserved in messagePayload
Malformed MIME	Caught; row stored with body = [PARSE_ERROR]
S3 object missing	NoSuchKey caught and logged; SQS message ignored (no retry storm)
Spam/abuse from a single sender	Rate-limited to 10 emails/minute per sender (in-memory Guava cache, per pod)

Existing institutes: rows in `email_address_mapping` are populated only when an email config is saved/re-saved going forward. To backfill, either ask each institute to re-save their email settings, or run a one-time SQL migration that copies addresses from the institute settings JSON.

Rollback

If something goes wrong after enabling:

1. Set `INBOUND_EMAIL_ENABLED=false` in GitHub secrets
2. Re-deploy

The listener stops, no further inbound emails are processed. Already-stored `INBOUND_EMAIL` rows remain in the DB (they're not harmful).

To fully revert the code, the V24 migration adds the `email_address_mapping` table — to drop it, write a V25 migration. The `Message-ID` capture in `EmailService` is purely additive and doesn't need rolling back.

File Reference

- `V24__Create_email_address_mapping.sql`
- `EmailAddressMapping.java`
- `EmailAddressMappingRepository.java`
- `AwsInboundConfig.java`
- `SqsInboundEmailListener.java`
- `InboundEmailService.java`
- `EmailService.java`
- `EmailConfigurationService.java`
- `CommunicationTimelineService.java`